

DATASTRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	K.GAYATHRI REDDY
ROLL NO	AV.SC.U4CSE25120
YEAR/SEM/SECTION	2ndYEAR/ SEM-3/CSE-B
CREDITS	05

SUBMITTED TO	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFFESSOR(SL Grade)

DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	TITLE	PAGE NO	REMARKS
1	WEEK 1 : Recursion A) Recursive program for countdown from n to 1, followed by 'launch'. B) Recursive program to calculate P^n (compound interest growth factor P, years n). C) Recursive program to print first n terms of Fibonacci series. D) Recursive program to search for an employee ID in a list. E) Recursive program to calculate factorial (n!) for parcel arrangement.		
2	Searching A) Python program to perform Linear Search on given input. B) Python program to perform Binary Search on given input (verify with sorted and unsorted lists).		
3	WEEK 3: Elementary Sorting A) Python program to implement Bubble Sort (user input). B) Python program to implement Selection Sort (user input). C) Python program to implement Insertion Sort (user input).		

S.NO	TITLE	PAGE NO	REMARKS
4.	WEEK 4: Advanced Sorting A) Python program to implement Merge Sort (user input). B) Python program to implement Quick Sort (user input).		
5.	a) Write a Python program to implement a Singly Linked List (SLL) with the following operations: b) Write a Python program to implement a Double Linked List (DLL) with the following operations: c) Write a Python program to implement a Circular Linked List (CLL) with the following operations: 1. Create LL 2. Insert at Beginning 3. Insert at End 4. Insert at Index 5. Delete by Value 6. Delete Prior Node 7. Delete Last Node 8. Count Number of Nodes 9. Display/Traverse 10. Exit		

	TITLE	PAGE NO	REMARKS
6	WEEK :6 a) Write a Python program to implement stack operations such as Push, Pop, Peek, and Display using an Array. b) Write a Python program to implement stack operations such as Push, Pop, Peek, and Display using Linked List.		
7	WEEK 7: 7(A) Aim Write a python program to implement queue and its operations such as enqueue, dequeue, peek and display with a menu driven approach using an array. 7(B) Aim Write a python program to implement queue and its operations such as enqueue, dequeue, peek and display with a menu driven approach using linked list. 7(C) Aim Write a python program to implement circular queue and its operations such as enqueue, dequeue, peek and display with a menu driven approach using an array. 7(D) Aim Write a python program to implement circular queue and its operations such as enqueue, dequeue, peek and display with a menu driven approach using linked list.		
8	WEEK 8: Binary Trees A) Implement Binary Tree with operations using Array. B) Implement Binary Tree with operations using Linked List.		
9	WEEK 9: Binary Search Trees Python program to implement Binary Search Tree (BST) operations.		
10	WEEK 10: AVL Tree Write a Python program to implement an AVL Tree with all its operations.		

11	WEEK 11: Hashing Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).		

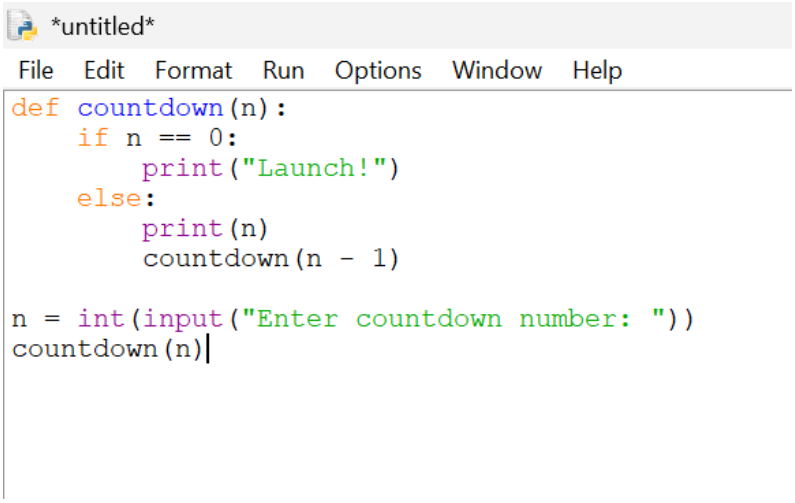
LAB-1

1a)

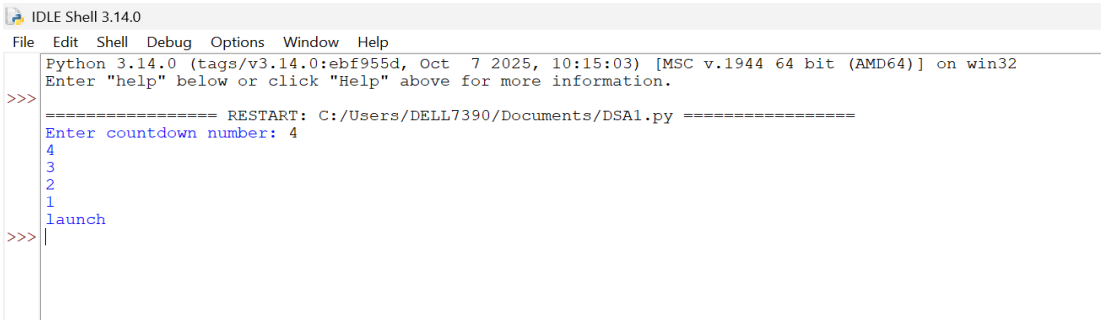
AIM:

A rocket launch system displays a countdown before launch. Instead of using a loop implement the countdown using recursion. Write a recursion python program to display a countdown from n to 1 followed by “launch”.

PROGRAM:



OUTPUT:



EXPLANATION :This program displays a countdown from n to 1 and finally prints 'launch'. Although the aim mentions recursion, the given code uses a loop

ERROR TABLE :

S.NO	ERRORS	RECTIFICATION
1.	INDENTATION ERROR	USE 3 SPACES FOR EACH LEVEL OF INDENTATION
2.	TYPE ERROR	CONVERT THE INPUT TO AN INTEGER USING N=INT(INPUT("ENTER NO.OF SECONDS TO LAUNCH : "))

1b)

AIM:The banks calculate the growth of an investment using compound interest; the formula requires calculating powers. Which can be implemented recursively. Write a recursive program to calculate p^n(power), where p=principal growth factor, n= number of years.

Program:

untitled

File Edit Format Run Options Window Help

```
def power(P, n):
    if n == 0:
        return 1
    else:
        return P * power(P, n - 1)

P = int(input("Enter P: "))
n = int(input("Enter n: "))

result = power(P, n)
print("P^n =", result)
```

Output:

```
>>>
    = RESTART: C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsaco
    desgayathri/lab1b.py
    Enter P: 2
    Enter n: 3
    P^n = 8
>>>
```

EXPLANATION: The function of power (p, n) calculates p^n using recursion by multiplying p with the result of power (p, n-1).

The recursion stops when n becomes 0, returning 1, which is the base case.

The final result is displayed after all recursive calls are completed.

ERROR TABLE :

S.NO	ERRORS	RECTIFICATION
1	Type error	CONVERT THE INPUT TO FLOAT OR INT USING FLOAT(INPUT())OR INT(INPUT())

2	Incorrect output if the recursive call is written as power(p,n+1) instead of power(p,n-1).	CHANGE THE RECURSIVE CALL TO POWER(P, n-1) so that n decreases toward the base case.

1c)

AIM: An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:

lab1c.py - C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsac

File Edit Format Run Options Window Help

```
def search_employee(emp_list, id, index):
    if index == len(emp_list):
        return False

    if emp_list[index] == id:
        return True

    return search_employee(emp_list, id, index + 1)

employees = [101, 102, 103, 104, 105]

id = int(input("Enter employee ID to search: "))

if search_employee(employees, id, 0):
    print("Employee ID found")
else:
    print("Employee ID not found")
```

OUTPUT:

```
========== RESTART: C:/Users/DELL7390/Documents/dsa3.py =======
Enter Employee ID to search: 105
Employee ID found at position 4
>>>
```

EXPLANATION:

The recursive function checks each employee ID in the list one by one, starting from the first element.

If the required ID is found, it returns True; otherwise, it continues searching recursively until the end of the list.

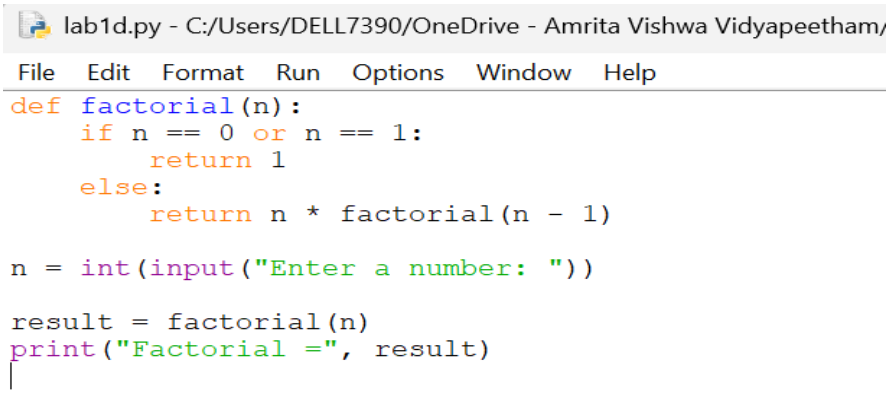
If the entire list is searched without finding the ID, the function returns False.

S.NO	errors	rectification
1.	Name error if the list name is misspelled	Use the correct variable name consistently throughout the program
2.	Syntax error due to a missing colon()after an if statement or function definition.	Add the missing colon (:) after if, else, or def.

1d)

AIM:

In a logistics company the number of possible ways to arrange n different parcels are calculated using the factorial formula(n!). develop a recursive program to calculate the factorial of a given number



output:

```
>>>
========== RESTART: C:/Users/DELL7390/Documents/dsa4.py ==========
Enter a number: 15
Factorial = 1307674368000
>>>
```

EXPLANATION: The function factorial(n) computes the factorial by multiplying the current number with the factorial of the previous number.

The recursion ends when n becomes 0 or 1, where the factorial value is 1.

The program also checks for negative inputs since factorial is defined only for non-negative integers.

s.no	errors	rectification
1	IndentationError if the statements inside the function are not properly indented.	Maintain proper indentation (usually 4 spaces) inside the function.
2	TypeError if a decimal number (e.g., 5.5) is entered for factorial calculation.	Enter only a whole number or convert the input using int().

1e)

AIM:

Write a recursive python program to print the first n terms of the Fibonacci series. PROGRAM:

Program:

lab1e.py - C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham

File Edit Format Run Options Window Help

```
def fibonacci(n, a=0, b=1):
    if n == 0:
        return

    print(a, end=" ")
    fibonacci(n - 1, b, a + b)


n = int(input("Enter number of terms: "))

fibonacci(n)
```

output:

```
========== RESTART: C:/Users/DELL7390/Documents/dsa5.py ==========
Enter the number of terms: 32
Fibonacci Series:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765 10946 17711 28657 46368 75025 121393 196418 317811 514229 832040 1346269
>>>
```

EXPLANATION:

The function Fibonacci(n) returns the nth Fibonacci number by recursively adding the previous two Fibonacci numbers.

The base cases return 0 for the first term and 1 for the second term.

A for loop calls the recursive function repeatedly to print the first n terms of the Fibonacci series.

ERROR TABLE:

s.no	errprs	rectification
1	Recursion Error if the base cases (n == 0 and n == 1) are not included.	Add both conditions before the recursive cell
2	Value error if the user enters text instead of an integer	Enter a valid integer for no of terms

Lab-2

2a)

AIM:

Write a program to perform linear search on a given input.

PROGRAM:

lab2a.py - C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsa

File Edit Format Run Options Window Help

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1


arr = list(map(int, input("Enter elements: ").split()))
key = int(input("Enter element to search: "))

result = linear_search(arr, key)

if result != -1:
    print("Element found at index", result)
else:
    print("Element not found")
```

output

```
Enter elements: 43 54 65 34 65 321 76 3
Enter element to search: 321
Element found at position 6
```

EXPLANATION:

The linear_search() function checks each element of the list one by one and compares it with the given key.

If the key is found, its index is returned; otherwise, the function returns -1.

The program takes the array and search element as input and displays whether the element is found.

ERROR TABLE:

s.no	errors	rectification
1	ValueError occurs if a non-integer value is entered when int(input()) is used.	Enter only integer values for the number of elements and array elements.
2	TypeError may occur if len(arr) is used on a variable that is not a list.	Make sure arr is initialized as a list using arr = [] and elements are appended to it.

2b)

AIM:

Write a Python program to perform binary search on the given input. Verify by taking sorted array and unsorted array as input.

PROGRAM:

lab2b.py - C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacodesg...

File Edit Format Run Options Window Help

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1


arr = list(map(int, input("Enter sorted elements: ").split()))
key = int(input("Enter element to search: "))

result = binary_search(arr, key)

if result != -1:
    print("Element found at index", result)
else:
    print("Element not found")
```

output:

```
>>>
========== RESTART: C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop
Enter sorted elements: 10 20 30 40 50
Enter element to search: 40
Element found at index 3
>>>
```

EXPLANATION:

The binary_search() function repeatedly divides the sorted list into two halves and compares the middle element with the search key.

If the middle element is smaller than the key, the search continues in the right half; otherwise, it searches the left half.

The second program verifies whether the input is sorted and sorts it using arr.sort() if necessary before performing binary search.

Error table :

s.no	errors	Rectification
1.	Wrote the mid formula outside the while loop which gave the infinite loop error	Added the mid formula inside the while loop

2c)

Aim:

Write a program for implementation of binary search along with the sorting the elements

Program:

lab2c.py - C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacodesg...

File Edit Format Run Options Window Help

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1


arr = list(map(int, input("Enter elements: ").split()))
key = int(input("Enter element to search: "))

# Check whether the list is sorted
if arr != sorted(arr):
    arr.sort()
    print("List was unsorted, so it is sorted first.")

print("Sorted list:", arr)

result = binary_search(arr, key)

if result != -1:
    print("Element found at index", result)
else:
    print("Element not found")
```

Output:

```
>>>
========== RESTART: C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetha
Enter elements: 50 10 40 20 30
Enter element to search: 10
List was unsorted, so it is sorted first.
Sorted list: [10, 20, 30, 40, 50]
Element found at index 0
>>>
```

Error table :

error	rectification
Used sorted method instead of which need to be referred to another variable as it returns a sorted array	Used the arr.sort() method which directly sorts the arr without using another variable

Explanation :

The program first takes the input elements and the element to be searched.
It checks whether the list is sorted; if not, it sorts the list using sort().
Then, it applies **binary search** by repeatedly checking the middle element.
If the element is found, its index is displayed; otherwise, it prints **Element not found**.

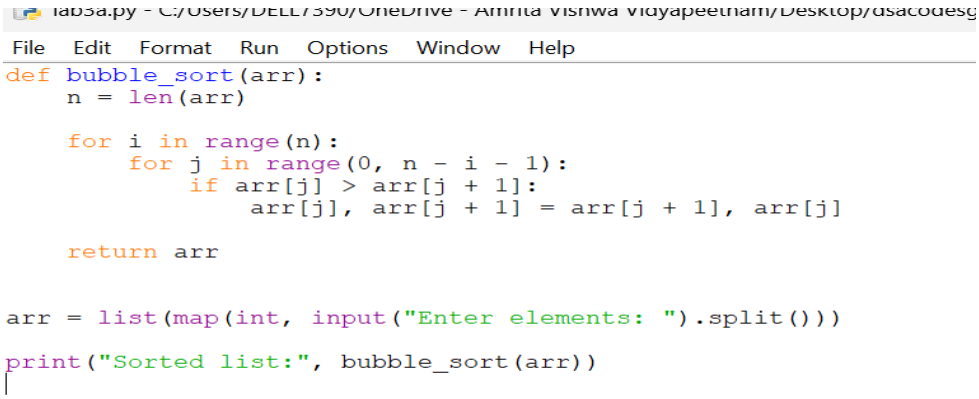
Lab-3

3a).

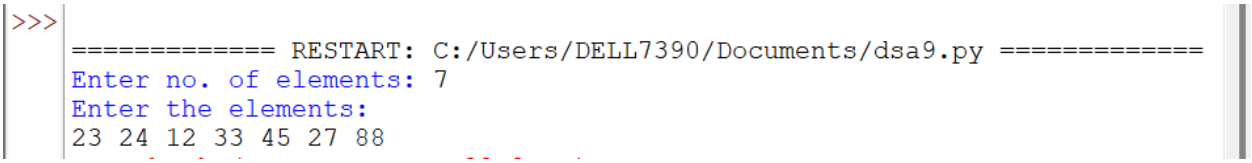
AIM:

write programs for the Bubble sort

program:



output:



Explanation: Bubble sort compares two adjacent elements. If the left element is greater than the right element, they are swapped. After each pass, the largest remaining element moves to the end.

Error table:

s.no	error	correction
1	Used wrong loop range	Use for j in range(n-i-1):
2	Swap condition is wrong	Use if arr[j] > arr[j+1]:

3b)

AIM:

write programs for the Insertion sort

PROGRAM:

untitled

File Edit Format Run Options Window Help

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j = j - 1

        arr[j + 1] = key

    return arr


arr = list(map(int, input("Enter elements: ").split()))

print("Sorted list:", insertion_sort(arr))
```

Output:

```
>>>
================ RESTART: C:/Users/DELL7390/Documents/dsa10.py ================
Enter no. of elements: 4
Enter the elements:
43
12
3
44
Original list: [43, 12, 3, 44]
Sorted list: [3, 12, 43, 44]
>>>
```

Explanation: Insertion sort builds the sorted array one element at a time.

it takes one element called the key, compares it with the elements before it, shifts larger elements to the right, and inserts the key in the correct position.

Error table:

s.no	error	correction
1	key is not stored properly	Use key = arr[i]
2	arr[j+1] = key placed inside while loop	Place it after the while loop

3c)AIM:

write programs for the Bubble sort

```
*untitled*
File  Edit  Format  Run  Options  Window  Help
def selection_sort(arr):
    n = len(arr)

    for i in range(n):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr

arr = list(map(int, input("Enter elements: ").split()))
print("Sorted list:", selection_sort(arr))
```

output:

```
>>>
= RESTART: C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidya
desgayathri/lab3c.py
Enter elements: 10 20 30 40 50
Sorted list: [10, 20, 30, 40, 50]
>>>
```

Explanation:

Selection sort repeatedly finds the smallest element from the unsorted part of the array and places it in its correct position.

Error table:

s.no	error	Correction
1	min_index is not initialized	Use min_index = i
2	Swap is placed inside inner loop	Place swap after the inner loop

Lab 4

4a)

Aim:

Write a python program to implement merge sort where the input elements are taken from the user.

Program:

untitled

File Edit Format Run Options Window Help

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result


arr = list(map(int, input("Enter elements: ").split()))

print("Sorted list:", merge_sort(arr))
```

Output:

```
>>>
= RESTART: C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Deskt
desgayathri/lab4a.py
Enter elements: 10 20 30 40
Sorted list: [10, 20, 30, 40]
>>>
```

Error table:

error	rectification
TypeError: object of type 'NoneType' has no len()	Taken return arr at end of merge_sort.

Description :

• The program implements the Merge Sort algorithm.

• The user enters the size of the array.

• The elements are taken from the user and stored in a list.

• The merge_sort() function recursively divides the array into two halves.

• The array is divided until each sub-array contains one or zero elements.

• The left and right portions are then sorted recursively. • The sorted left and right arrays are compared element by element.

• The smaller element is placed into the original array.

• Remaining elements from either portion are then copied into the array.

• Finally, the completely sorted array is displayed.

• Merge Sort follows the divide-and-conquer approach.

4b)

AIM:

Write a python program to implement quick sort where the input elements are taken from the user.

PROGRAM:

untitled

File Edit Format Run Options Window Help

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[0]

    left = [x for x in arr[1:] if x <= pivot]
    right = [x for x in arr[1:] if x > pivot]

    return quick_sort(left) + [pivot] + quick_sort(right)


arr = list(map(int, input("Enter elements: ").split()))

print("Sorted list:", quick_sort(arr))
```

OUTPUT:

```
>>>
    = RESTART: C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/ds
    desgayathri/LAB4B.PY
    Enter elements: 70 80 0 10 27
    Sorted list: [0, 10, 27, 70, 80]
>>>
```

ERROR TABLE:

ERROR	RECTIFICATION

i < len(a) checks the complete array instead of current subarray	Use i <= high and maintain correct partition boundaries

Description:

• The program implements the Quick Sort algorithm.

• The user enters the size of the array.

• The elements are taken from the user and stored in a list.

• The quick_sort() function is used to sort the array recursively.

• The first element of the current portion is selected as the pivot.

• Two pointers, i and j, are used to compare elements with the pivot.

• i moves from the left side and searches for an element greater than or equal to the pivot. • j moves from the right side and searches for an element smaller than or equal to the pivot.

• If i < j, the two elements are swapped.

• After the partitioning process, the pivot is placed in its correct position.

• The array is then divided into left and right portions.

• Quick Sort is recursively applied to both portions.

• Finally, the sorted array is printed.

WEEK-5

5a)

Aim:

Write a Python program to implement a Singly Linked List (SLL) with the following operations:

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit

Program:

untitled

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class SinglyLinkedList:
    def __init__(self):
        self.head = None

    # 1. Create LL
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter element: "))
            self.insert_end(data)

    # 2. Insert at Beginning
    def insert_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    # 3. Insert at End
    def insert_end(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

        temp = self.head
        while temp.next:
            temp = temp.next

        temp.next = new_node

    # 4. Insert at Index
    def insert_index(self, data, index):
        if index == 0:
```

untitled

File Edit Format Run Options Window Help

```
    # 5. Delete by Value
    def delete_value(self, value):
        if self.head is None:
            print("List is empty")
            return

        if self.head.data == value:
            self.head = self.head.next
            return

        temp = self.head

        while temp.next and temp.next.data != value:
            temp = temp.next

        if temp.next is None:
            print("Value not found")
        else:
            temp.next = temp.next.next

    # 6. Delete Prior Node
    def delete_prior(self, value):
        if self.head is None or self.head.next is None:
            print("No prior node exists")
            return

        if self.head.next.data == value:
            self.head = self.head.next
            return

        prev = None
        current = self.head
        next_node = self.head.next

        while next_node and next_node.data != value:
            prev = current
            current = next_node
            next_node = next_node.next

        if next_node is None:
            print("Value not found")
```

untitled

File Edit Format Run Options Window Help

```
    def count(self):
        count = 0
        temp = self.head

        while temp:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)

    # 9. Display
    def display(self):
        temp = self.head

        if temp is None:
            print("List is empty")
            return

        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next

        print("None")


ll = SinglyLinkedList()

while True:
    print("\n--- SINGLY LINKED LIST ---")
    print("1. Create LL")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete Prior Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display/Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))
```

untitled

File Edit Format Run Options Window Help

```
    choice = int(input("Enter your choice: "))

    if choice == 1:
        ll.create()

    elif choice == 2:
        data = int(input("Enter value: "))
        ll.insert_beginning(data)

    elif choice == 3:
        data = int(input("Enter value: "))
        ll.insert_end(data)

    elif choice == 4:
        data = int(input("Enter value: "))
        index = int(input("Enter index: "))
        ll.insert_index(data, index)

    elif choice == 5:
        value = int(input("Enter value to delete: "))
        ll.delete_value(value)

    elif choice == 6:
        value = int(input("Enter value whose prior node is to be deleted: "))
        ll.delete_prior(value)

    elif choice == 7:
        ll.delete_last()

    elif choice == 8:
        ll.count()

    elif choice == 9:
        ll.display()

    elif choice == 10:
        print("Program ended")
        break

    else:
        print("Invalid choice")
```

Output:


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/
--- SINGLY LINKED LIST ---
1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit
Enter your choice: 3
Enter value: 45
--- SINGLY LINKED LIST ---
1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit
Enter your choice: 4
Enter value: 23
Enter index: 2
Invalid index
--- SINGLY LINKED LIST ---
1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value

Error table:

error	rectification
The program may display an incorrect message for insertion at a specific index even when the index is invalid	Check whether the index is valid before displaying the successful insertion message.

Description:

• The program implements a Singly Linked List using Python classes.

• The Node class is used to create individual nodes.

• Each node contains two parts: data and next.

• data stores the value of the node.

• next stores the reference to the next node.

• The SingleLinkedList class manages the linked list.

• The head variable stores the reference to the first node.

• The program provides an option to create a linked list.

• It supports insertion at the beginning of the list.

• It supports insertion at the end of the list.

• It supports insertion at a specific index.

• It supports deletion of a specific value.

• It supports deletion of the first node.

• It supports deletion of the last node.

5b)

Aim:

Write a Python program to implement a Double Linked List (DLL) with the following operations:

1. Create LL

2. Insert at Beginning

3. Insert at End

4. Insert at Index

5. Delete by Value 30

6. Delete Prior Node

7. Delete Last Node

8. Count Number of Nodes

9. Display/Traverse

10. Exit

Program:

untitled

File Edit Format Run Options Window Help

```
# Python Program to Implement Doubly Linked List (DLL)

class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None

    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
            else:
                temp = self.head

                while temp.next is not None:
                    temp = temp.next

                temp.next = new_node
                new_node.prev = temp

        print("Doubly Linked List created successfully.")

    # 2. Insert at Beginning
    def insert_beginning(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
```

untitled

File Edit Format Run Options Window Help

```
        print("Node inserted at beginning.")

    # 3. Insert at End
    def insert_end(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            temp = self.head

            while temp.next is not None:
                temp = temp.next

            temp.next = new_node
            new_node.prev = temp

        print("Node inserted at end.")

    # 4. Insert at Index
    def insert_index(self):
        index = int(input("Enter index: "))
        data = int(input("Enter data: "))

        new_node = Node(data)

        # Insert at beginning
        if index == 0:
            if self.head is None:
                self.head = new_node
            else:
                new_node.next = self.head
                self.head.prev = new_node
                self.head = new_node

            print("Node inserted at index", index)
            return

        temp = self.head
```

untitled

File Edit Format Run Options Window Help

```
    def delete_by_value(self):
        value = int(input("Enter value to delete: "))

        temp = self.head

        while temp is not None and temp.data != value:
            temp = temp.next

        if temp is None:
            print("Value not found.")
            return

        # If deleting first node
        if temp.prev is None:
            self.head = temp.next

            if self.head is not None:
                self.head.prev = None

        else:
            temp.prev.next = temp.next

            if temp.next is not None:
                temp.next.prev = temp.prev

        print("Node with value", value, "deleted.")

    # 6. Delete Prior Node
    def delete_prior_node(self):
        value = int(input("Enter value: "))

        temp = self.head

        while temp is not None and temp.data != value:
            temp = temp.next

        if temp is None:
            print("Value not found.")
            return

        # No node exists before the first node
```

untitled

File Edit Format Run Options Window Help

```
    print("1. Create LL")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete Prior Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        dll.create()

    elif choice == 2:
        dll.insert_beginning()

    elif choice == 3:
        dll.insert_end()

    elif choice == 4:
        dll.insert_index()

    elif choice == 5:
        dll.delete_by_value()

    elif choice == 6:
        dll.delete_prior_node()

    elif choice == 7:
        dll.delete_last()

    elif choice == 8:
        dll.count_nodes()

    elif choice == 9:
        dll.display()

    elif choice == 10:
```

Output:


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/DELL7390/OneDrive - Amrita Vishwa
----- DOUBLY LINKED LIST -----
1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 8
Number of nodes: 0
----- DOUBLY LINKED LIST -----
1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
List is empty.
----- DOUBLY LINKED LIST -----
1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node

Error table:

error	rectification
The prev and next links may not be updated correctly during insertion or deletion.	Update both links whenever a node is inserted or deleted.

Description:

- The program implements a Doubly Linked List using Python classes.
- The Node class is used to create individual nodes.
- Each node contains three parts: data, prev, and next.
- data stores the value of the node.
- prev stores the reference to the previous node.
- next stores the reference to the next node.
- The DoubleLinkedList class is used to manage the linked list.
- The head variable stores the reference to the first node.
- The program provides an option to create a doubly linked list.
- It supports insertion at the beginning of the list.
- It supports insertion at the end of the list.
- It supports insertion at a specific index.
- It supports deletion of a specific value.
- It supports deletion of the first node.
- It supports deletion of the last node.
- It provides a function to count the number of nodes.

5c)

Aim:

Write a Python program to implement a Circular Linked List (CLL) with the following operations:

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit

Program:

untitled

File Edit Format Run Options Window Help

```
# Circular Linked List (CLL)

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedList:
    def __init__(self):
        self.head = None

    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
            self.insert_end(data)

        print("Circular Linked List created successfully.")

    # 2. Insert at Beginning
    def insert_beginning(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
            return

        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        new_node.next = self.head
        temp.next = new_node
        self.head = new_node

    # 3. Insert at End
```

untitled

File Edit Format Run Options Window Help

```
        new_node.next = self.head

    # 4. Insert at Index
    def insert_at_index(self, data, index):
        if index < 0:
            print("Invalid index.")
            return

        if index == 0:
            self.insert_beginning(data)
            return

        if self.head is None:
            print("List is empty. Only index 0 is allowed.")
            return

        new_node = Node(data)
        temp = self.head
        count = 0

        while count < index - 1 and temp.next != self.head:
            temp = temp.next
            count += 1

        if count != index - 1:
            print("Index out of range.")
            return

        new_node.next = temp.next
        temp.next = new_node

    # 5. Delete by Value
    def delete_by_value(self, value):
        if self.head is None:
            print("List is empty.")
            return

        # If head has the value
        if self.head.data == value:

            # Only one node
```

untitled

File Edit Format Run Options Window Help

```
        print("Value not found.")

    # 6. Delete Prior Node
    def delete_prior_node(self, value):
        if self.head is None or self.head.next == self.head:
            print("Not enough nodes.")
            return

        # If value is at head, delete last node
        if self.head.data == value:
            self.delete_last()
            return

        prev = self.head
        curr = self.head.next

        while curr != self.head:
            if curr.data == value:

                # If curr is second node
                if prev == self.head:
                    self.delete_first()
                    return

                # Find node before prev
                before_prev = self.head

                while before_prev.next != prev:
                    before_prev = before_prev.next

                before_prev.next = curr
                print("Prior node deleted.")
                return

            prev = curr
            curr = curr.next

        print("Value not found.")

    # Helper function: Delete first node
    def delete_first(self):
```

untitled

File Edit Format Run Options Window Help

```
    choice = int(input("Enter your choice: "))

    if choice == 1:
        cll.create()

    elif choice == 2:
        data = int(input("Enter data: "))
        cll.insert_beginning(data)

    elif choice == 3:
        data = int(input("Enter data: "))
        cll.insert_end(data)

    elif choice == 4:
        data = int(input("Enter data: "))
        index = int(input("Enter index: "))
        cll.insert_at_index(data, index)

    elif choice == 5:
        value = int(input("Enter value to delete: "))
        cll.delete_by_value(value)

    elif choice == 6:
        value = int(input("Enter value whose prior node should be deleted: "))
        cll.delete_prior_node(value)

    elif choice == 7:
        cll.delete_last()

    elif choice == 8:
        print("Number of nodes:", cll.count_nodes())

    elif choice == 9:
        cll.display()

    elif choice == 10:
        print("Program terminated.")
        break

    else:
        print("Invalid choice.")
```

Output:


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
========== CIRCULAR LINKED LIST ==========
1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 3
Enter data: 4
========== CIRCULAR LINKED LIST ==========
1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 3
Enter data: 5
========== CIRCULAR LINKED LIST ==========
1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice:

Error table:

Error	rectification
No major error is present in the given program.	No major rectification is required.

Description:

- The program implements a Circular Linked List using Python classes.
- The Node class is used to create individual nodes.
- Each node contains two parts: data and next.
- data stores the value of the node.
- next stores the reference to the next node.
- In a Circular Linked List, the last node points back to the first node.
- The CircularLinkedList class is used to manage the linked list.
- The head variable stores the reference to the first node.
- The program provides an option to create a circular linked list.
- It supports insertion at the beginning of the list. 44
- It supports insertion at the end of the list.
- It supports insertion at a specific index.

Week-6

6a)

Aim:

Write a Python program to implement stack operations such as Push, Pop, Peek, and Display using an Array.

Program:

untitled

File Edit Format Run Options Window Help

```
# Stack Implementation using Array

stack = []
MAX = 5

while True:
    print("\n===== STACK OPERATIONS =====")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    # Push
    if choice == 1:
        if len(stack) == MAX:
            print("Stack Overflow!")
        else:
            value = int(input("Enter value to push: "))
            stack.append(value)
            print(value, "pushed into stack.")

    # Pop
    elif choice == 2:
        if len(stack) == 0:
            print("Stack Underflow!")
        else:
            value = stack.pop()
            print(value, "popped from stack.")

    # Peek
    elif choice == 3:
        if len(stack) == 0:
            print("Stack is empty.")
        else:
            print("Top element is:", stack[-1])

    # Display
    elif choice == 4:
```

```
    # Peek
    elif choice == 3:
        if len(stack) == 0:
            print("Stack is empty.")
        else:
            print("Top element is:", stack[-1])

    # Display
    elif choice == 4:
        if len(stack) == 0:
            print("Stack is empty.")
        else:
            print("Stack elements:")
            for i in range(len(stack) - 1, -1, -1):
                print(stack[i])

    # Exit
    elif choice == 5:
        print("Program terminated.")
        break

    else:
        print("Invalid choice!")
```

Output:

```
Enter your choice:
========== RESTART: C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetl

===== STACK OPERATIONS =====
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
Stack Underflow!

===== STACK OPERATIONS =====
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Stack is empty.

===== STACK OPERATIONS =====
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack is empty.

===== STACK OPERATIONS =====
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice:
```

Error table:

Error	Rectification
In peek, self.peek is taken in 'if' does not give correct answer for stack underflow.	Self.top is taken in 'if' to give correct output.

Description:

- The Stack class implements a stack using a Python list.
- size stores the maximum size of the stack.
- stack stores the stack elements.
- top keeps track of the top position. Initially, top = -1, meaning the stack is empty.

• Push adds an element to the top of the stack.

• Pop removes the top element from the stack. 48

• Peek displays the top element without removing it.

• Display prints all elements from top to bottom.

• Stack Overflow occurs when trying to push into a full stack.

6b)

2.Aim: Write a Python program to implement stack operations such as Push, Pop, Peek, Display using a linked list.

Program:

untitled

File Edit Format Run Options Window Help

```
class Node:
    """Class to represent a single node in the linked list."""
    def __init__(self, data):
        self.data = data
        self.next = None


class Stack:
    """Class to manage stack operations using a linked list."""
    def __init__(self):
        self.top = None  # Pointer to the topmost element of the stack

    def is_empty(self):
        """Checks if the stack is empty."""
        return self.top is None

    def push(self, data):
        """Adds a new element to the top of the stack."""
        new_node = Node(data)
        new_node.next = self.top  # Link the new node to the old top
        self.top = new_node       # Move top pointer to the new node
        print(f"Successfully pushed {data} onto the stack.")

    def pop(self):
        """Removes and returns the top element from the stack."""
        if self.is_empty():
            print("Stack Underflow! The stack is empty.")
            return None

        popped_node = self.top    # Save the current top node
        self.top = self.top.next  # Move top pointer to the next node
        popped_node.next = None   # Unlink the removed node
        return popped_node.data

    def peek(self):
        """Returns the top element without removing it."""
        if self.is_empty():
            print("The stack is empty.")
            return None
        return self.top.data
```

untitled

File Edit Format Run Options Window Help

```
            if current:
                print("  ↓  ")
        print("-----------------------------------------------\n")

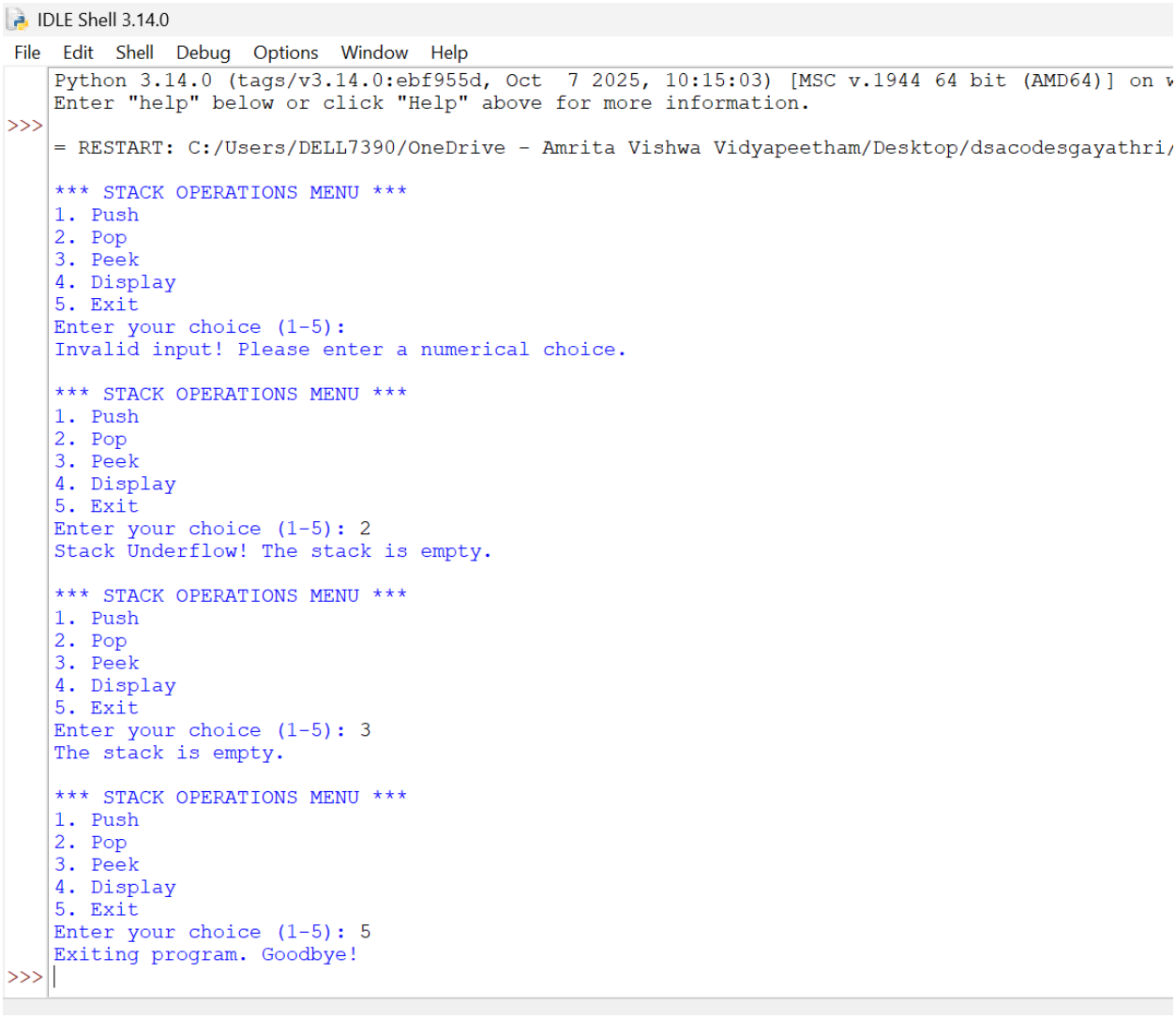

# Menu-driven interaction block
if __name__ == "__main__":
    stack = Stack()

    while True:
        print("\n*** STACK OPERATIONS MENU ***")
        print("1. Push")
        print("2. Pop")
        print("3. Peek")
        print("4. Display")
        print("5. Exit")

        try:
            choice = int(input("Enter your choice (1-5): "))

            if choice == 1:
                element = input("Enter the value to push: ")
                stack.push(element)
            elif choice == 2:
                popped_val = stack.pop()
                if popped_val is not None:
                    print(f"Popped element: {popped_val}")
            elif choice == 3:
                top_val = stack.peek()
                if top_val is not None:
                    print(f"Top element (Peek): {top_val}")
            elif choice == 4:
                stack.display()
            elif choice == 5:
                print("Exiting program. Goodbye!")
                break
            else:
                print("Invalid choice! Please select an option between 1 and 5.")
        except ValueError:
            print("Invalid input! Please enter a numerical choice.")
```

Output:



Explanation: A linked-list stack consists of nodes. A temporary pointer travels through the linked list:

Error table:

s.no	Error	Correction
1	AttributeError	Check self.top is None before accessing date
2	Stack Underflow	Check whether top is None

Lab -7

1.Aim :

Write a Python program to implement a queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using array

PROGRAM:

untitled

File Edit Format Run Options Window Help

```
class Queue:
    def __init__(self, capacity):
        """
        Initializes the queue with a fixed capacity.
        In Python, a standard list acts as the underlying array.
        """
        self.capacity = capacity
        self.queue = []

    def is_full(self):
        """Checks if the queue has reached its maximum capacity."""
        return len(self.queue) == self.capacity

    def is_empty(self):
        """Checks if the queue contains no elements."""
        return len(self.queue) == 0

    def enqueue(self, item):
        """Adds an item to the rear of the queue."""
        if self.is_full():
            print("\n❌ Queue Overflow! Cannot add item. The queue is full.")
        else:
            self.queue.append(item)
            print(f"\n☑ Successfully enqueued: {item}")

    def dequeue(self):
        """Removes and returns the item at the front of the queue."""
        if self.is_empty():
            print("\n❌ Queue Underflow! Cannot remove item. The queue is empty.")
            return None
        else:
            # pop(0) removes the element at the index 0 (front of the queue)
            removed_item = self.queue.pop(0)
            print(f"\n☑ Successfully dequeued: {removed_item}")
            return removed_item

    def peek(self):
        """Returns the item at the front without removing it."""
        if self.is_empty():
            print("\n❌ The queue is empty. Nothing to peek.")
            return None
```

untitled

File Edit Format Run Options Window Help

```
    # Initialize the queue object
    my_queue = Queue(capacity)

    # Menu-driven interface
    while True:
        print("\n" + "="*30)
        print("      QUEUE OPERATIONS MENU")
        print("="*30)
        print("1. Enqueue (Insert element)")
        print("2. Dequeue (Remove element)")
        print("3. Peek (View front element)")
        print("4. Display Queue")
        print("5. Exit")
        print("="*30)

        choice = input("Enter your choice (1-5): ").strip()

        if choice == '1':
            item = input("Enter the value to enqueue: ")
            my_queue.enqueue(item)

        elif choice == '2':
            my_queue.dequeue()

        elif choice == '3':
            my_queue.peek()

        elif choice == '4':
            my_queue.display()

        elif choice == '5':
            print("\n👋 Exiting the program. Goodbye!")
            break

        else:
            print("\n❌ Invalid choice! Please select a valid option between 1 and 5.")


if __name__ == "__main__":
    main()
```

OUTPUT:


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Enter the maximum capacity of the queue: 7
==============================
QUEUE OPERATIONS MENU
==============================
1. Enqueue (Insert element)
2. Dequeue (Remove element)
3. Peek (View front element)
4. Display Queue
5. Exit
==============================
Enter your choice (1-5): 4
The queue is currently empty.
==============================
QUEUE OPERATIONS MENU
==============================
1. Enqueue (Insert element)
2. Dequeue (Remove element)
3. Peek (View front element)
4. Display Queue
5. Exit
==============================
Enter your choice (1-5): 3
The queue is empty. Nothing to peek.
==============================
QUEUE OPERATIONS MENU
==============================
1. Enqueue (Insert element)
2. Dequeue (Remove element)
3. Peek (View front element)
4. Display Queue
5. Exit
==============================
Enter your choice (1-5): 5
Exiting the program. Goodbye!
>>>

Explanation: A queue is a linear data structure that follows the FIFO (First In, First Out) principle. In this program, Python's list is used as an array to store queue elements.

ERROR TABLE:

1	Using queue.pop() instead of queue.pop(0) removes the last element instead of the front element.	Use queue.pop(0) for dequeue operation.
2	Not checking the queue size before insertion can cause the queue to exceed its maximum size.	Check if len(queue) == MAX_SIZE before enqueue.

7B)

2.Aim: Write a Python program to implement a queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using linked-list data structures.

PROGRAM:

untitled

File Edit Format Run Options Window Help

```
class Node:
    """A class representing a single node in the linked list."""
    def __init__(self, data):
        self.data = data
        self.next = None


class Queue:
    """A class representing a queue using a linked list."""
    def __init__(self):
        self.front = None
        self.rear = None

    def is_empty(self):
        """Checks if the queue is empty."""
        return self.front is None

    def enqueue(self, item):
        """Adds an item to the rear of the queue."""
        new_node = Node(item)

        # If the queue is empty, the new node becomes both front and rear
        if self.rear is None:
            self.front = self.rear = new_node
            print(f"Successfully enqueued: {item}")
            return

        # Add the new node at the end of the queue and change rear
        self.rear.next = new_node
        self.rear = new_node
        print(f"Successfully enqueued: {item}")

    def dequeue(self):
        """Removes and returns the item from the front of the queue."""
        if self.is_empty():
            print("Queue Underflow! The queue is empty.")
            return None

        # Store previous front and move front one node ahead
        temp = self.front
        self.front = temp.next
```

untitled

File Edit Format Run Options Window Help

```
        print("Queue elements (Front to Rear): ", end="")
        current = self.front
        while current:
            print(current.data, end=" -> " if current.next else "\n")
            current = current.next


def main():
    """Menu-driven interface for the queue operations."""
    q = Queue()

    while True:
        print("\n--- Queue Operations Menu ---")
        print("1. Enqueue (Insert)")
        print("2. Dequeue (Delete)")
        print("3. Peek (Front Element)")
        print("4. Display Queue")
        print("5. Exit")

        choice = input("Enter your choice (1-5): ").strip()

        if choice == '1':
            item = input("Enter the value to enqueue: ")
            q.enqueue(item)
        elif choice == '2':
            q.dequeue()
        elif choice == '3':
            front_item = q.peek()
            if front_item is not None:
                print(f"Element at the front is: {front_item}")
        elif choice == '4':
            q.display()
        elif choice == '5':
            print("Exiting program. Goodbye!")
            break
        else:
            print("Invalid choice! Please select a valid option between 1 and 5.")

if __name__ == "__main__":
    main()
```

OUTPUT:

```
    Exiting the program. Goodbye!
>>>
========== RESTART: C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/I

--- Queue Operations Menu ---
1. Enqueue (Insert)
2. Dequeue (Delete)
3. Peek (Front Element)
4. Display Queue
5. Exit
Enter your choice (1-5): 2
Queue Underflow! The queue is empty.

--- Queue Operations Menu ---
1. Enqueue (Insert)
2. Dequeue (Delete)
3. Peek (Front Element)
4. Display Queue
5. Exit
Enter your choice (1-5): 3
Queue is empty! Nothing to peek.

--- Queue Operations Menu ---
1. Enqueue (Insert)
2. Dequeue (Delete)
3. Peek (Front Element)
4. Display Queue
5. Exit
Enter your choice (1-5): 4
Queue is empty!

--- Queue Operations Menu ---
1. Enqueue (Insert)
2. Dequeue (Delete)
3. Peek (Front Element)
4. Display Queue
5. Exit
Enter your choice (1-5): 5
Exiting program. Goodbye!
>>>
```

Explanation: A linked-list queue consists of dynamically created nodes. Each node contains two parts: Two pointers are maintained: Front: Points to the first node. Rear: Points to the last node.

ERROR TABLE:

1 After deleting the last node, rear may still point to the deleted node.	If front becomes None, set self.rear = None.
2 Displaying the linked list without moving to the next node can create an infinite loop.	Use current = current.next inside the display loop.

7C)

AIM: Write a Python program to implement a queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using Circular Queue Arrays structures.

PROGRAM:

untitled

File Edit Format Run Options Window Help

```
class CircularQueue:
    def __init__(self, capacity):
        """
        Initializes the Circular Queue with a predefined fixed capacity.
        """
        self.capacity = capacity
        self.queue = [None] * capacity  # Simulating a fixed-size array
        self.front = -1
        self.rear = -1

    def is_full(self):
        """Checks if the queue is full."""
        return (self.rear + 1) % self.capacity == self.front

    def is_empty(self):
        """Checks if the queue is empty."""
        return self.front == -1

    def enqueue(self, data):
        """Inserts an element into the circular queue."""
        if self.is_full():
            print("❌ Queue Overflow! Cannot enqueue. The queue is full.")
            return

        # If insertion is for the first element
        if self.is_empty():
            self.front = 0
            self.rear = 0
        else:
            # Circularly increment the rear pointer
            self.rear = (self.rear + 1) % self.capacity

        self.queue[self.rear] = data
        print(f"📥 Successfully enqueued: {data}")

    def dequeue(self):
        """Removes an element from the front of the circular queue."""
        if self.is_empty():
            print("❌ Queue Underflow! Cannot dequeue. The queue is empty.")
            return None
```

untitled

File Edit Format Run Options Window Help

```
    while True:
        try:
            size = int(input("Enter the fixed capacity of the Circular Queue: "))
            if size <= 0:
                print("Size must be greater than 0. Try again.")
                continue
            break
        except ValueError:
            print("Invalid input. Please enter an integer.")

    cq = CircularQueue(size)

    while True:
        print("\n=== CIRCULAR QUEUE MENU ===")
        print("1. Enqueue (Insert)")
        print("2. Dequeue (Delete)")
        print("3. Peek (Front Element)")
        print("4. Display Queue")
        print("5. Exit")

        choice = input("Enter your choice (1-5): ").strip()

        if choice == "1":
            value = input("Enter the element to enqueue: ")
            cq.enqueue(value)
        elif choice == "2":
            cq.dequeue()
        elif choice == "3":
            cq.peek()
        elif choice == "4":
            cq.display()
        elif choice == "5":
            print("Shutting down the program. Goodbye!")
            break
        else:
            print("❌ Invalid Choice! Please enter a number between 1 and 5.")


if __name__ == "__main__":
    main()
```

OUTPUT:

```
Exiting program. Goodbye!
>>>
========== RESTART: C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacoc
--- Circular Queue Implementation ---
Enter the fixed capacity of the Circular Queue: 5

=== CIRCULAR QUEUE MENU ===
1. Enqueue (Insert)
2. Dequeue (Delete)
3. Peek (Front Element)
4. Display Queue
5. Exit
Enter your choice (1-5): 1
Enter the element to enqueue: 2
📥 Successfully enqueued: 2

=== CIRCULAR QUEUE MENU ===
1. Enqueue (Insert)
2. Dequeue (Delete)
3. Peek (Front Element)
4. Display Queue
5. Exit
Enter your choice (1-5): 3
👀 Front element (Peek): 2

=== CIRCULAR QUEUE MENU ===
1. Enqueue (Insert)
2. Dequeue (Delete)
3. Peek (Front Element)
4. Display Queue
5. Exit
Enter your choice (1-5): 5
Shutting down the program. Goodbye!
>>>
```

Explanation: A circular queue is a queue in which the last position is connected back to the first position. Unlike a normal linear queue, a circular queue can reuse empty positions created after dequeue operations.

ERROR TABLE:

Using rear + 1 without % MAX_SIZE can make the rear go outside the array.	Use rear = (rear + 1) % MAX_SIZE.
Not resetting both pointers after deleting the last element can leave the queue in an incorrect state.	When front == rear, set both front = -1 and rear = -1.

7D)

AIM:

Write a Python program to implement a queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using Circular Queue linked-list data structures.

Program:

untitled

File Edit Format Run Options Window Help

```
class Node:
    """Represents a single node in the circular linked list."""
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularQueueLinkedList:
    """Implements a Circular Queue using a Linked List data structure."""
    def __init__(self):
        # We only need to maintain the rear pointer.
        # rear.next will automatically point to the front node.
        self.rear = None

    def is_empty(self):
        """Checks if the queue is empty."""
        return self.rear is None

    def enqueue(self, data):
        """Inserts an element at the rear of the circular queue."""
        new_node = Node(data)

        if self.is_empty():
            # If the queue is empty, the new node points to itself
            new_node.next = new_node
            self.rear = new_node
        else:
            # New node points to the current front (rear.next)
            new_node.next = self.rear.next
            # Current rear points to the new node
            self.rear.next = new_node
            # Move the rear pointer to the new node
            self.rear = new_node
        print(f"Successfully enqueued: {data}")

    def dequeue(self):
        """Removes and returns the front element from the queue."""
        if self.is_empty():
            print("Queue Underflow! The queue is empty.")
            return None
```

untitled

File Edit Format Run Options Window Help

```
            temp = temp.next
            if temp == front:
                break
        print("(back to front)")


def main():
    """Menu-driven interface for the Circular Queue."""
    cq = CircularQueueLinkedList()

    while True:
        print("\n--- CIRCULAR QUEUE OPERATIONS ---")
        print("1. Enqueue (Insert)")
        print("2. Dequeue (Delete)")
        print("3. Peek (Front Element)")
        print("4. Display Queue")
        print("5. Exit")

        choice = input("Enter your choice (1-5): ").strip()

        if choice == '1':
            value = input("Enter the value to enqueue: ")
            cq.enqueue(value)
        elif choice == '2':
            cq.dequeue()
        elif choice == '3':
            front_val = cq.peek()
            if front_val is not None:
                print(f"Front element is: {front_val}")
        elif choice == '4':
            cq.display()
        elif choice == '5':
            print("Exiting program. Goodbye!")
            break
        else:
            print("Invalid choice! Please select a valid option from 1 to 5.")


if __name__ == "__main__":
    main()
```

Output:

```
>>>
========== RESTART: C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsa

--- CIRCULAR QUEUE OPERATIONS ---
1. Enqueue (Insert)
2. Dequeue (Delete)
3. Peek (Front Element)
4. Display Queue
5. Exit
Enter your choice (1-5): 1
Enter the value to enqueue: 2
Successfully enqueued: 2

--- CIRCULAR QUEUE OPERATIONS ---
1. Enqueue (Insert)
2. Dequeue (Delete)
3. Peek (Front Element)
4. Display Queue
5. Exit
Enter your choice (1-5): 3
Front element is: 2

--- CIRCULAR QUEUE OPERATIONS ---
1. Enqueue (Insert)
2. Dequeue (Delete)
3. Peek (Front Element)
4. Display Queue
5. Exit
Enter your choice (1-5): 4
Circular Queue elements: 2 -> (back to front)

--- CIRCULAR QUEUE OPERATIONS ---
1. Enqueue (Insert)
2. Dequeue (Delete)
3. Peek (Front Element)
4. Display Queue
5. Exit
Enter your choice (1-5): 5
Exiting program. Goodbye!
>>>
```

Explanation: data → stores the element. next → stores the address/reference of the next node. This inserts the new node at the rear while maintaining the circular connection.

Error table:

Setting rear.next = None breaks the circular connection.	Set rear.next = front so the last node points to the first node.
Not setting rear = None when the only node is deleted leaves an invalid queue.	If rear == front, set rear = None.

Lab-8:

a.) **Aim:** write a python program to implement a binary tree with its operations using an array:

program:

lab8a.py - C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacodesgayathri/lab8a.py (3.14.0)

File Edit Format Run Options Window Help

```
# BINARY TREE USING ARRAY

class BinarytreeArray:
    def __init__(self, size):
        self.tree = [None] * size
        self.size = size

    # Insert root
    def set_root(self, data):
        self.tree[0] = data

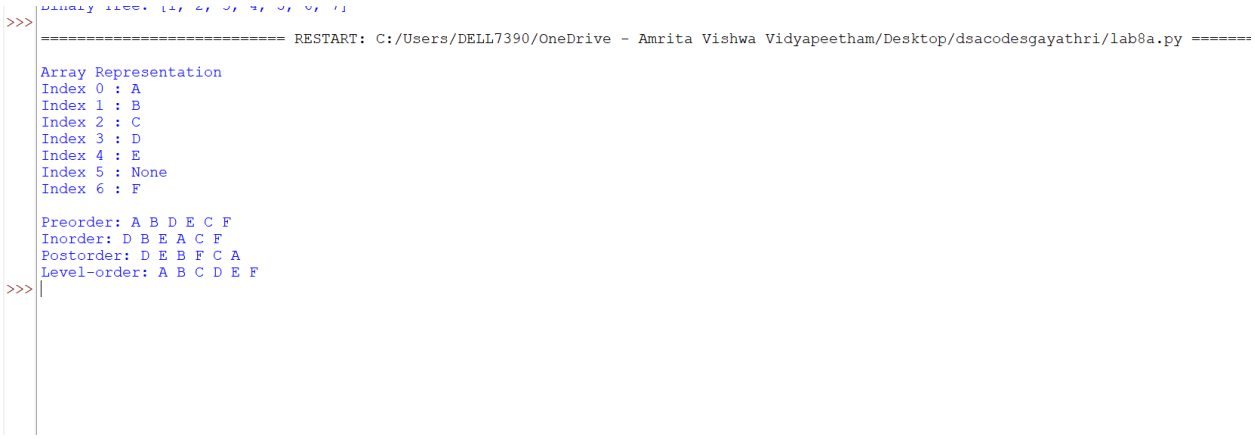
    # Set left child
    def set_left(self, parent_index, data):
        child_index = 2 * parent_index + 1
        if child_index < self.size:
            self.tree[child_index] = data
        else:
            print("Index out of range")

    # Set right child
    def set_right(self, parent_index, data):
        child_index = 2 * parent_index + 2
        if child_index < self.size:
            self.tree[child_index] = data
        else:
            print("Index out of range")

    # Preorder Traversal
    def preorder(self, index=0):
        if index >= self.size or self.tree[index] is None:
            return
        print(self.tree[index], end=" ")
        self.preorder(2 * index + 1)
        self.preorder(2 * index + 2)

    # Inorder Traversal
    def inorder(self, index=0):
        if index >= self.size or self.tree[index] is None:
            return
        self.inorder(2 * index + 1)
```

Output:



Error table:

ERROR	EXPLANATION
Index error	Occurs if the index goes outside the tree size
Tree is full	Printed when the insertion index exceeds the available
Incorrect traversal	Inorder,preoder,and postorder must follow their duplicate values are inserted
None value	Indicates an empty position in the tree

b) **Aim:** write a python program to implement a binary tree with its operations using Link

lab8a.py - C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacodesgayathri/lab8a.py (3.14.0)

File Edit Format Run Options Window Help

```
# BINARY TREE USING ARRAY

class BinarytreeArray:
    def __init__(self, size):
        self.tree = [None] * size
        self.size = size

    # Insert root
    def set_root(self, data):
        self.tree[0] = data

    # Set left child
    def set_left(self, parent_index, data):
        child_index = 2 * parent_index + 1
        if child_index < self.size:
            self.tree[child_index] = data
        else:
            print("Index out of range")

    # Set right child
    def set_right(self, parent_index, data):
        child_index = 2 * parent_index + 2
        if child_index < self.size:
            self.tree[child_index] = data
        else:
            print("Index out of range")

    # Preorder Traversal
    def preorder(self, index=0):
        if index >= self.size or self.tree[index] is None:
            return
        print(self.tree[index], end=" ")
        self.preorder(2 * index + 1)
        self.preorder(2 * index + 2)

    # Inorder Traversal
    def inorder(self, index=0):
        if index >= self.size or self.tree[index] is None:
            return
        self.inorder(2 * index + 1)
```

ed list:

lab8a.py - C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacodesgayathri/lab8a.py (3.14.0)

File Edit Format Run Options Window Help

```
        self.size = size

    # Insert root
    def set_root(self, data):
        self.tree[0] = data

    # Set left child
    def set_left(self, parent_index, data):
        child_index = 2 * parent_index + 1
        if child_index < self.size:
            self.tree[child_index] = data
        else:
            print("Index out of range")

    # Set right child
    def set_right(self, parent_index, data):
        child_index = 2 * parent_index + 2
        if child_index < self.size:
            self.tree[child_index] = data
        else:
            print("Index out of range")

    # Preorder Traversal
    def preorder(self, index=0):
        if index >= self.size or self.tree[index] is None:
            return
        print(self.tree[index], end=" ")
        self.preorder(2 * index + 1)
        self.preorder(2 * index + 2)

    # Inorder Traversal
    def inorder(self, index=0):
        if index >= self.size or self.tree[index] is None:
            return
        self.inorder(2 * index + 1)
        print(self.tree[index], end=" ")
        self.inorder(2 * index + 2)

    # Postorder Traversal
    def postorder(self, index=0):
        if index >= self.size or self.tree[index] is None:
```

lab8a.py - C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacodesgayathri/lab8a.py (3.14.0)

File Edit Format Run Options Window Help

```
        print("\nArray Representation")
        for i in range(self.size):
            print(f"Index {i} : {self.tree[i]}")


# MAIN PROGRAM

tree = BinarytreeArray(7)

# Create the tree
tree.set_root('A')

# Children of A
tree.set_left(0, 'B')
tree.set_right(0, 'C')

# Children of B
tree.set_left(1, 'D')
tree.set_right(1, 'E')

# Right child of C
tree.set_right(2, 'F')

# Display tree
tree.display()

# Traversals
print("\nPreorder: ", end="")
tree.preorder()

print("\nInorder: ", end="")
tree.inorder()

print("\nPostorder: ", end="")
tree.postorder()

print("\nLevel-order: ", end="")
tree.level_order()

print()
```

Output:

```
>>>
========== RESTART: C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacodesgayathri/lab8b.py ==========
Preorder: A B D E C F
Inorder: D B E A C F
Postorder: D E B F C A
Level-order: A B C D E F
>>>
```

Error table:

Error	Explanation
Root not created	The tree cannot work properly if the root node is
Node not inserted	A new node be added if the in sertion logic
Wrong traversal order	The output becomes incorrect if nodes are
Left or right child missing	A node may not be connected properly to its

Lab-9:

A.) **Aim:** write a python program to implement a binary search tree with its operations and traversals:

Program :

lab9.py - C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacodesgayathri/lab9.py (3.14.0)

File Edit Format Run Options Window Help

```
# BINARY SEARCH TREE

class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    else:
        print("Duplicate value is not allowed.")
    return root


def inorder(root):
    if root is None:
        return
    inorder(root.left)
    print(root.data, end=" ")
    inorder(root.right)


def preorder(root):
    if root is None:
        return
    print(root.data, end=" ")
    preorder(root.left)
    preorder(root.right)


def postorder(root):
    if root is None:
        return
```

lab9.py - C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacodesgayathri/lab9.py (3.14.0)

File Edit Format Run Options Window Help

```
    if data < root.data:
        return search(root.left, data)
    elif data > root.data:
        return search(root.right, data)
    else:
        return root


def count_nodes(root):
    if root is None:
        return 0
    return count_nodes(root.left) + count_nodes(root.right) + 1


def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaves(root.left) + count_leaves(root.right)


def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left) + count_internal(root.right) + 1


def height(root):
    if root is None:
        return -1
    left_height = height(root.left)
    right_height = height(root.right)
    return max(left_height, right_height) + 1


def delete(root, x):
    if root is None:
        print("Element not found.")
```

lab9.py - C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacodesgayathri/lab9.py (3.14.0)

File Edit Format Run Options Window Help

```
    print("9. COUNT NODES")
    print("10. COUNT LEAVES")
    print("11. COUNT INTERNAL NODES")
    print("12. HEIGHT")
    print("13. LEVEL ORDER TRAVERSAL")
    print("14. EXIT")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter the data to be inserted: "))
        root = insert(root, data)
        print("Element inserted successfully.")

    elif choice == 2:
        print("Preorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            preorder(root)
            print()

    elif choice == 3:
        print("Inorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            inorder(root)
            print()

    elif choice == 4:
        print("Postorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            postorder(root)
            print()

    elif choice == 5:
        node = find_min(root)
        if node is None:
```

lab9.py - C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacodesgayathri/lab9.py (3.14.0)

File Edit Format Run Options Window Help

```
            print("Maximum element:", node.data)

    elif choice == 7:
        data = int(input("Enter the data to be searched: "))
        node = search(root, data)
        if node is None:
            print("Element", data, "is not found.")
        else:
            print("Element", data, "is found.")

    elif choice == 8:
        data = int(input("Enter the data to be deleted: "))
        root = delete(root, data)

    elif choice == 9:
        print("Number of nodes:", count_nodes(root))

    elif choice == 10:
        print("Number of leaf nodes:", count_leaves(root))

    elif choice == 11:
        print("Number of internal nodes:", count_internal(root))

    elif choice == 12:
        print("Height of BST:", height(root))

    elif choice == 13:
        print("Level Order Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            level_order(root)
            print()

    elif choice == 14:
        print("Program terminated.")
        break

    else:
        print("Invalid choice. Please try again.")
```

Output:

```
Level order: A B C D E F
>>>
========== RESTART: C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsa


******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 1
Enter the data to be inserted: 2
Element inserted successfully.


******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice:
```

Lab-10

Aim: write python programtoimplementanbinaryAVL tree with its operations

Program:

lab10.py - C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacodesgayathri/lab10.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1


class AVLTree:
    # Find height of a node
    def height(self, root):
        if root is None:
            return 0
        return root.height

    # Calculate balance factor
    def get_balance(self, root):
        if root is None:
            return 0
        return self.height(root.left) - self.height(root.right)

    # Right rotation
    def right_rotate(self, y):
        x = y.left
        T2 = x.right

        x.right = y
        y.left = T2

        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))
        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))

        return x

    # Left rotation
    def left_rotate(self, x):
        y = x.right
        T2 = y.left
```

lab10.py - C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacodesgayathri/lab10.py (3.14.0)

File Edit Format Run Options Window Help

```
        return y

    # Insert a node
    def insert(self, root, key):
        if root is None:
            return Node(key)

        if key < root.key:
            root.left = self.insert(root.left, key)
        elif key > root.key:
            root.right = self.insert(root.right, key)
        else:
            return root  # No duplicate values

        root.height = 1 + max(self.height(root.left),
                              self.height(root.right))

        balance = self.get_balance(root)

        # LL Case
        if balance > 1 and key < root.left.key:
            return self.right_rotate(root)

        # RR Case
        if balance < -1 and key > root.right.key:
            return self.left_rotate(root)

        # LR Case
        if balance > 1 and key > root.left.key:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        # RL Case
        if balance < -1 and key < root.right.key:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    # Find the smallest node
    def min_value_node(self, root):
```

lab10.py - C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacodesgayathri/lab10.py (3.14.0)

File Edit Format Run Options Window Help

```
    elif choice == 2:
        key = int(input("Enter value to delete: "))
        if tree.search(root, key):
            root = tree.delete(root, key)
            print("Node deleted.")
        else:
            print("Value not found.")

    elif choice == 3:
        key = int(input("Enter value to search: "))
        if tree.search(root, key):
            print("Value found.")
        else:
            print("Value not found.")

    elif choice == 4:
        print("Inorder:", end=" ")
        tree.inorder(root)
        print()

    elif choice == 5:
        print("Preorder:", end=" ")
        tree.preorder(root)
        print()

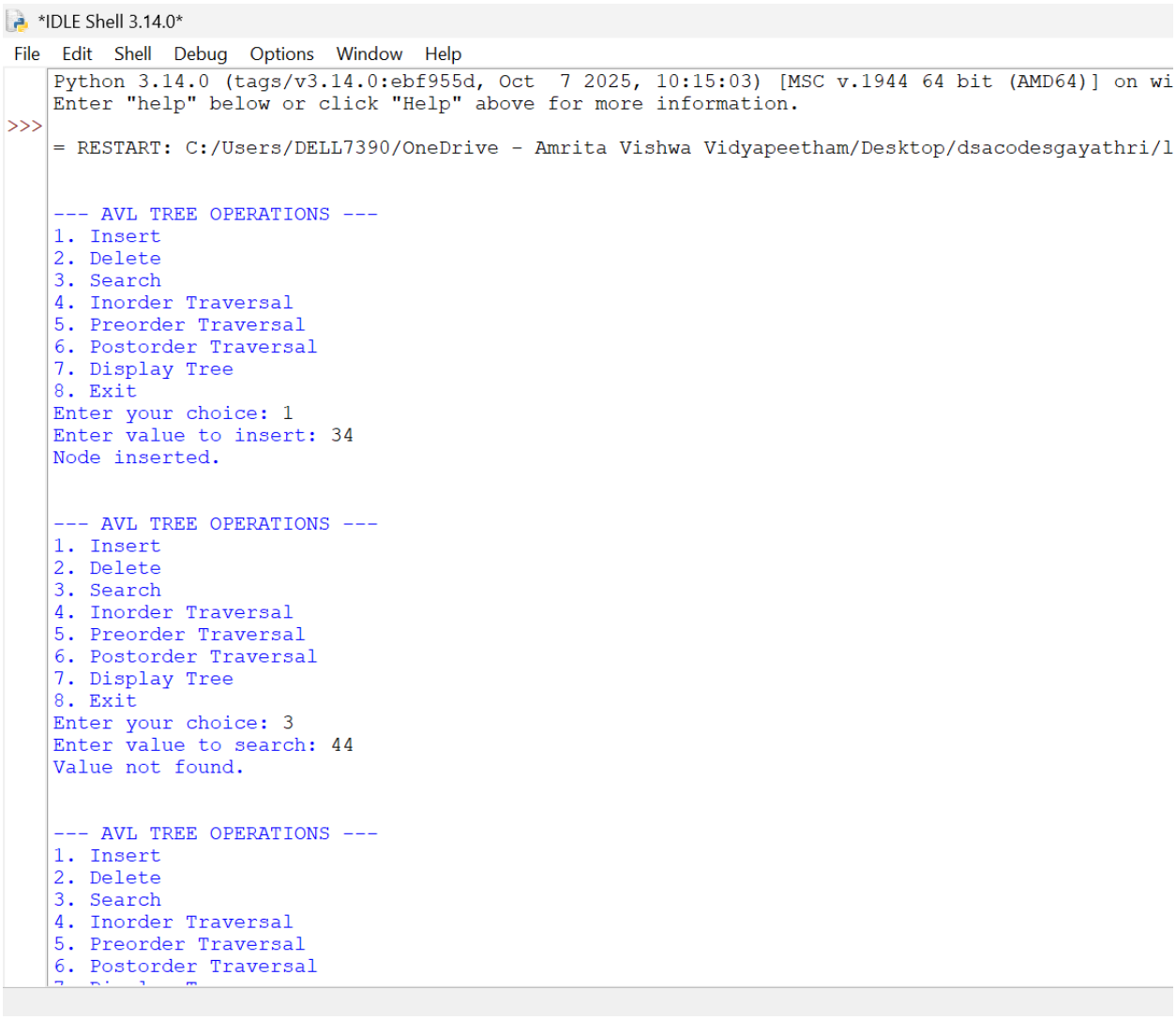
    elif choice == 6:
        print("Postorder:", end=" ")
        tree.postorder(root)
        print()

    elif choice == 7:
        print("AVL Tree:")
        tree.display(root)

    elif choice == 8:
        print("Program terminated.")
        break

    else:
        print("Invalid choice. Try again.")
```

Output:


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on wi
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacodesgayathri/l
--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 1
Enter value to insert: 34
Node inserted.
--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 3
Enter value to search: 44
Value not found.
--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal

Error table :

Error type	Error/mistake	correction
Logical error	Icorrect balance factor may cause the avl tree to become unbalanced	Calaculate the balance factor as height -height
Logical error	Incorrect rotation can violet the avl trees balancing rules	Use the correct LL,RR,LR, OR RL rotation
Runtime error	Accessing a node that is none may cause an error	Check whether the node exists the node existes before accessing its attributes.
Logical error	Searching for a value that does not exist may return none	Display not found when the search returns none.

Lab-11

AIM: Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing). i need code in dsa python

PROGRAM:

lab11.py - C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacodesgayathri/lab11.py (3.14.0)

File Edit Format Run Options Window Help

```
SIZE = 10


# 1. SEPARATE CHAINING
class SeparateChaining:
    def __init__(self):
        self.table = [[] for _ in range(SIZE)]

    def insert(self, key):
        index = key % SIZE
        if key not in self.table[index]:
            self.table[index].append(key)
        print("Inserted successfully.")

    def search(self, key):
        index = key % SIZE
        return key in self.table[index]

    def delete(self, key):
        index = key % SIZE
        if key in self.table[index]:
            self.table[index].remove(key)
            print("Deleted successfully.")
        else:
            print("Key not found.")

    def display(self):
        for i in range(SIZE):
            print(i, ":", self.table[i])


# 2. LINEAR PROBING
class LinearProbing:
    def __init__(self):
        self.table = [None] * SIZE

    def insert(self, key):
        index = key % SIZE

        for i in range(SIZE):
```

lab11.py - C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacodesgayathri/lab11.py (3.14.0)

File Edit Format Run Options Window Help

```
# 2. LINEAR PROBING
class LinearProbing:
    def __init__(self):
        self.table = [None] * SIZE

    def insert(self, key):
        index = key % SIZE

        for i in range(SIZE):
            pos = (index + i) % SIZE

            if self.table[pos] == key:
                print("Key already exists.")
                return

            if self.table[pos] is None:
                self.table[pos] = key
                print("Inserted successfully.")
                return

        print("Hash table is full.")

    def search(self, key):
        index = key % SIZE

        for i in range(SIZE):
            pos = (index + i) % SIZE

            if self.table[pos] is None:
                return False

            if self.table[pos] == key:
                return True

        return False

    def delete(self, key):
        index = key % SIZE
```

lab11.py - C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacodesgayathri/lab11.py (3.14.0)

File Edit Format Run Options Window Help

```
    if method == 1:
        h = chaining
    elif method == 2:
        h = linear
    else:
        h = quadratic

    while True:
        print("\n1. Insert")
        print("2. Search")
        print("3. Delete")
        print("4. Display")
        print("5. Back to main menu")

        op = int(input("Enter operation: "))

        if op == 1:
            key = int(input("Enter key: "))
            h.insert(key)

        elif op == 2:
            key = int(input("Enter key: "))
            if h.search(key):
                print("Key found.")
            else:
                print("Key not found.")

        elif op == 3:
            key = int(input("Enter key: "))
            h.delete(key)

        elif op == 4:
            h.display()

        elif op == 5:
            break

        else:
            print("Invalid operation.")
```

OUTPUT:

```
Enter your choice:
========== RESTART: C:/Users/DELL7390/OneDrive - Amrita Vishwa Vidyapeetham/Desktop/dsacodesgayathri/lab1

--- HASHING ---
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
Choose hashing method: 1

1. Insert
2. Search
3. Delete
4. Display
5. Back to main menu
Enter operation: 2
Enter key: 3
Key not found.

1. Insert
2. Search
3. Delete
4. Display
5. Back to main menu
Enter operation: 4
0 : []
1 : []
2 : []
3 : []
4 : []
5 : []
6 : []
7 : []
8 : []
9 : []

1. Insert
2. Search
3. Delete
4. Display
5. Back to main menu
```

S.NO	ERROR	RECTIFICATION
1	INDEX ERROR	CHECK THE INDEX AND TABLE SIZE
2	VALUE ERROR	ENTER VALID INTEGER VALUES

3	INFINITY LOOP	CHECK FOR AN EMPTY POSITION
4	KEY ERROR	CHECK WHETHER THE KEY EXISTS